

Verificació de Programes

ITERATIUS

Què vol dir "DEBILITAR" una assertió?

La idea és que fem que sigui més genèrica, és a dir, que sigui certa més vegades.

Sigui A una assertió.

Sigui A' el debilitament d' A .

$$A \Rightarrow A'$$

però no necessàriament tenim que:

$$A' \Rightarrow A.$$

Can Sebilitem?

La idea és eliminar una conjunció:

$$A \wedge P \rightsquigarrow A$$

$$A \wedge P \Rightarrow A$$

però no necessàriament :

$$A \Rightarrow A \wedge P$$

(només cert quan P és cert).

El que forem és:

$$A \Rightarrow A \wedge P \rightsquigarrow A$$

debitament

o també:

$$A \Rightarrow A \wedge P \rightsquigarrow A \wedge P'$$

debitament

(on $P \Rightarrow P'$).

Exemple:

$$A = S = \sum_{i=0}^{N-1} v[i]$$

Afegim una nova variable

$$S = \sum_{i=0}^{j-1} v[i] \quad \wedge \quad j = N$$

equivalent

i debilitem eliminant (o consistint) una clàusula:

$$A' \quad S = \sum_{i=0}^{j-1} v[i] \quad \wedge \quad j \leq N$$

debilitada.

(fixeu-vos que $j=N \Rightarrow j \leq N$, però $j \leq N \not\Rightarrow j=N$).

Per a verificar un programa iteratiu cal trobar un

INVARIANT

Què és un invariant?

- (1) És una assertió lògica. És a dir, avalua a CERT/FALS.
- (2) Ho d'avaluar a cert durant l'execució del bucle, i just abans, i just després de l'execució del bucle.

- (3) (i més important) ho s'explica l'estat de les variables que hi ha a dins del bucle (les que canvien, òbviament).

Que hi té a veure el debilitament i
l'invariant?

Moltes vegades (no vol dir sempre!!) l'invariant
és el debilitament de la postcondició.

int summa (vector<int> v)

PRE $\equiv$

int s = 0;

int j = 0;

int N = v.size();

INV $\equiv$??

while (j < N) {

 s = s + v[j];

 j = j + 1;

}

return s;

POST $\equiv$ $\left\{ s = \sum_{i=0}^{N-1} v[i] \right\}$

int summa (vector<int> v)

PRE $\equiv$

```
int s = 0;
int j = 0;
int N = v.size();
```

INV $\equiv$

$$s = \sum_{i=0}^{j-1} v[i] \wedge j \leq N$$

while (j < N) {

 s = s + v[j];

 j = j + 1;

}

↑ debilitament

return s;

$$s = \sum_{i=0}^{N-1} v[i]$$

}

$\equiv$

$$s = \sum_{i=0}^{j-1} v[i] \wedge j = N$$

+ equivalència

POST $\equiv$


```
int sumo(vector<int> v)
```

```
PRE =
```

```
int s = 0;
```

```
int j = 0;
```

```
int N = v.size();
```

```
INV =
```

```
while (j < N)
```

```
{ s = s + v[j];
```

```
j = j + 1;
```

```
}
```

```
return s;
```

$s = \sum_{i=0}^{j-1} v[i] \wedge j \leq N$

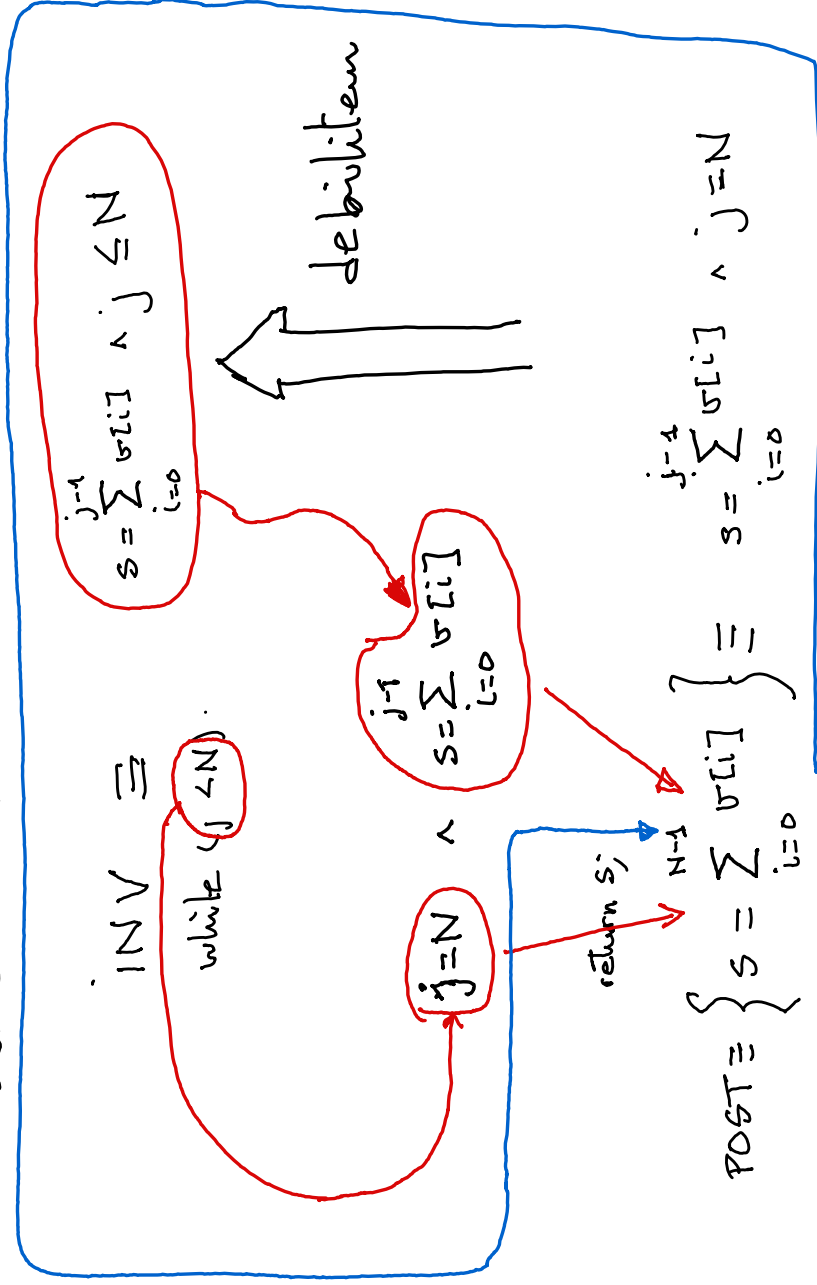
debutitem

$\sum_{i=0}^{j-1} v[i] = s$

return s;

POST = $\left\{ s = \sum_{i=0}^{N-1} v[i] \right\} = s = \sum_{i=0}^{j-1} v[i] \wedge j = N$

Hem debilitat la POST per a obtenir un INV que, un cop acabat el bucle, i conjuntament amb la negació de la condició del bucle, ens torna la post.



Això NO és una fórmula exacta:

de vegades no tenim directament la post,
i cal fer algunes instruccions.

Però ens pot donar una idea de
quin és l'INV.

Calculer l'algorithme d'une pile.

int algorithme(stack <int> p)

PRE $\equiv p = P$

$n = \emptyset$;

while (not p.empty()) {

$n = n + 1$;

 p.pop();

return n;

INV $\equiv n = |P| - |p|$

debilement
eliminant
 $|p| = \emptyset$



POST $\equiv n = |P| \equiv n = |P| - |p| \wedge |p| = \emptyset$

Verifiquem:

1) INV es compleix abans del bucle:

$$\left. \begin{array}{l} PRE \equiv P = p \\ n = 0; \end{array} \right\} \Rightarrow |P| = |p| \Rightarrow |P| - |p| = 0 \quad \updownarrow (n=0)$$

$$INV \equiv n = |P| - |p| \quad \checkmark$$

2 L'INV est mainte après de quatr iteratio

$$INV \equiv n = |P| - |p|$$

while (not p.empty()) {

$n = n + 1;$

$p.pop();$

$$n + 1 = |P| - |p|$$

|||

$$n + 1 = |P| - (|p| - 1)$$

|||

$$INV \equiv n = |P| - |p| \equiv n + 1 = |P| - |p| + 1$$

Q

3. Després del bucle (+instructions, si n'hi ha quessim)
arribem a la post.

while (not p.empty()) {

$n = n + 1;$

 p.pop();

}

INV $\equiv n = |P| - |p| \wedge p.empty()$

|||

$n = |P| - \phi$ (axioma $|p-bucle| = \phi$)

|||

POST $\equiv n = |P|$ ✓

4. Funció definida: $f(p, n) = |P|^{-n}$

(a) $f(p, n) \in \mathbb{N}$: n va de 0 fins a $|P|$, per tant $|P|^{-n} \in \mathbb{N}$ (definida en els naturals).

(b) decreix estrictament a cada iteració:

$$|P|^{-n} > |P|^{-(n+1)} = |P|^{-n-1}$$

Gerador en un vector.

```
bool cerca (vector<int> v, int e)
PRE ≡ cont
    bool trobat = false;
    int i = 0;
    int N = v.size();
```

```
    while (i < N and not trobat) {
```

```
        trobat = v[i] == e;
```

```
        i = i + 1;
```

$$\text{trobat} = \bigvee_{j=0}^{i-1} v[j] = e \wedge i \leq N$$



```
    return b;
```

debititem

$$\text{POST} \equiv \text{trobat} = \bigvee_{j=0}^{N-1} v[j] = e \Rightarrow \text{trobat} = \bigvee_{j=0}^{i-1} v[j] = e \wedge i = N$$

1. L'invariant es compleix abans d'entrar al bucle.

bool `trobot` = false;

$$\text{trobot} = \bigvee_{j=0}^{i-1} v[j] = e \wedge i \leq N$$

int `i` = ϕ ;

$$\text{false} = \bigvee_{j=0}^{\phi-1} v[j] = e \wedge \phi \leq N$$

int `N` = `v.size()`;

$$\text{false} = \bigvee_{j=0}^{\phi-1} v[j] = e \wedge \phi \leq v.\text{size}()$$

cert: donem un buit,
implica el neutre
de l'operació: false

cert pq
 $v.\text{size}() \geq \phi$.

2. L' invariant es manté a cada iteració.

$$trobot = \bigvee_{j=0}^{i-1} v[j] = e \wedge i \leq N$$

$$\downarrow \quad \text{trobot} = v[i] = e$$

$$trobot = \bigvee_{j=0}^{i-1} v[j] = e \vee v[i] = e \wedge i \leq N$$

$$\downarrow \text{ajuntarem } \exists$$

$$trobot = \bigvee_{j=0}^i v[j] = e \wedge i \leq N$$

$$\downarrow$$

$$i = i + 1$$

$$trobot = \bigvee_{j=0}^i v[j] = e \wedge i + 1 \leq N$$

$$\uparrow$$

$$\forall i < N$$

segons la condició.

while ($i < N$ and not trobot) {

 trobot = $v[i]$ == e;

$i = i + 1$;

$$trobot = \bigvee_{j=0}^{i-1} v[j] = e \wedge i \leq N$$

2. Después de salir del bucle, arribem a la POST.

$$INV \equiv b = \bigvee_{j=0}^{i-1} v[j] = e \wedge i \leq N$$

while $(i < N \text{ and not } \text{trobats}) \{$

$\text{trobats} = v[i] == e;$

$i = i + 1;$

negació de la condició + INV

$$\neg (i < N \wedge \neg \text{trobats})$$

$$\equiv (i = N \vee \text{trobats}) \wedge \text{trobats} = \bigvee_{j=0}^{i-1} v[j] = e \wedge i \leq N$$

Negació de la condició

$$\neg (j < N \wedge \neg \text{trobat})$$

$$\stackrel{\text{de Morgan}}{=} (j = N \vee \text{trobat})$$

$$\left[(j = N \wedge \neg \text{trobat}) \vee \text{trobat} \right]$$

$$\wedge \text{INV} \Rightarrow \text{POST}$$

$$\wedge \text{trobat} = \bigvee_{j=0}^{i-1} \neg \sigma[j] = e \wedge i \leq N$$

apliquem distributivitat

$$i = N \wedge \neg \text{trobat} = \bigvee_{j=0}^{i-1} \neg \sigma[j] = e \wedge i \leq N \vee \text{trobat} = \bigvee_{j=0}^{i-1} \neg \sigma[j] = e \wedge i \leq N$$

$$i = N \wedge \neg \text{trobat} = \bigvee_{j=0}^{N-1} \neg \sigma[j] = e$$

$$\vee \text{trobat} = \bigvee_{j=0}^{i-1} \neg \sigma[j] = e \wedge i \leq N$$

Si apliquem distributivitat múltiple:

$$(P \wedge Q) \vee (R \wedge S) = (P \vee R) \wedge (P \vee S) \wedge (Q \vee R) \wedge (Q \vee S)$$

$$P \wedge Q \quad \vee \quad R \wedge S$$

$$i=N \wedge \text{trobat} = \bigvee_{j=0}^{N-1} v[i][j] = e \quad \vee \quad \text{trobat} = \bigvee_{j=0}^{i-1} v[i][j] = e \wedge i \leq N$$

$$P \quad Q \quad R \quad S$$

$$P \wedge R \quad \vee \quad Q \wedge S$$

$$i=N \wedge \text{trobat} = \bigvee_{j=0}^{i-1} v[i][j] = e$$

$$\Downarrow$$

$$i=N \wedge \text{trobat} = \bigvee_{j=0}^{i-1} v[i][j] = e$$

$$i=N \wedge \text{trobat} = \bigvee_{j=0}^{i-1} v[i][j] = e$$

Função de fila.

$$f(i) = N - i$$

(a) Com que $i \leq N$, abstruindo $N - i \in \mathbb{N}$

$$(b) \quad N - i > N - (i + 1) = N - i - 1$$

$$\uparrow$$
$$i = i + 1$$

Partición.

vector, not

partir (vector a, not x)

$$PFE \cong \emptyset \leq i \leq j+1 < |a|$$

$$PFE \cong \emptyset \leq i \leq k \leq j \leq i+1 < |a|$$

$$PFE \cong \emptyset \leq i \leq k \leq j \leq i+1 < |a|$$

